

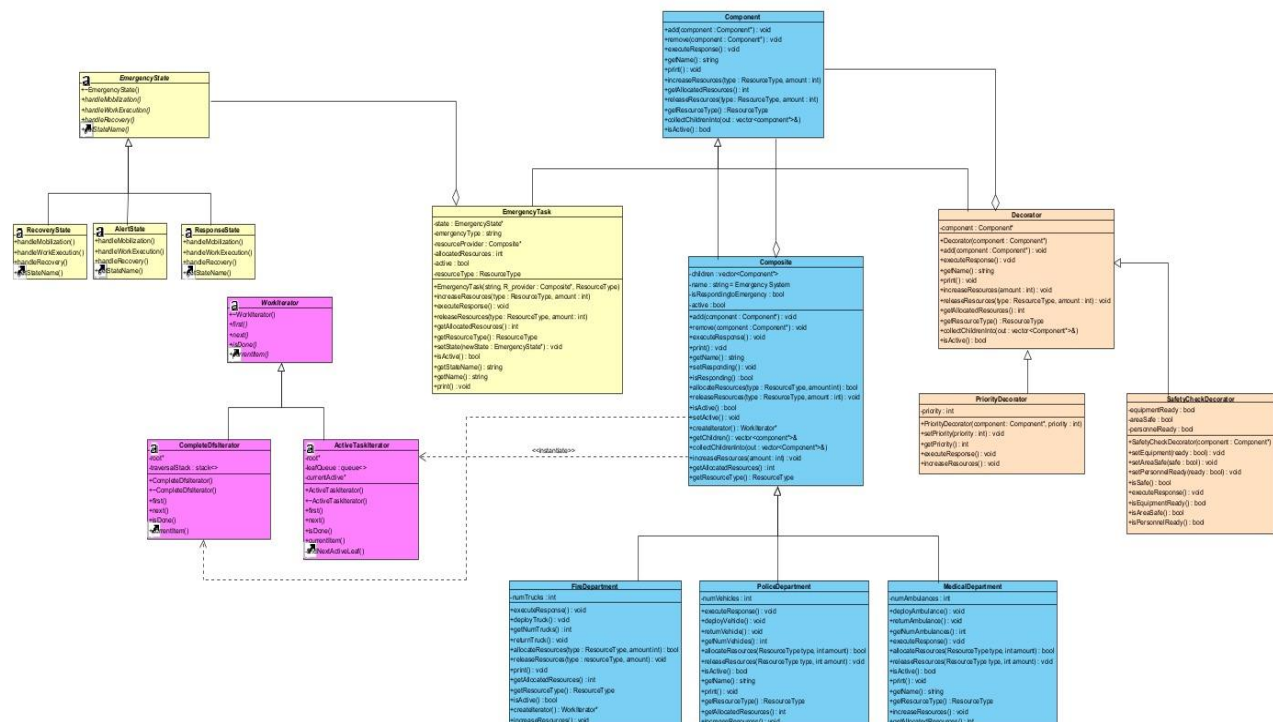
Emergency Response Management System

Dingalethu Ngumbela – u25170547
Ronin Sydney –u25056507
Name – Student Number

Task 1

System description

This system organises emergency response work into a clear hierarchy: an incident breaks down into departments, departments into units, and units into individual tasks. It tracks each task through its lifecycle; from the moment it's assigned all the way through to resolution and flags any attempt to perform an action that doesn't make sense for the current stage. Coordinators can also filter down to only the most critical active tasks at any point, rather than wading through everything at once. The goal is simple: make sure nothing falls through the cracks during an emergency.



Pattern Participants

<i>Pattern</i>	<i>GoF Participant</i>	<i>TaskForge Participant</i>
State:	Context	EmergencyTask
	State	EmergencyState
	Concrete States	AlertState, ResponseState, RecoveryState
Composite:	Component	Component
	Composite	Composite
	Leaf	EmergencyTask
	Client	main()
Decorator:	Component	Component
	Concrete Component	EmergencyTask
	Decorator	WorkDecorator
	Concrete Decorator	PriorityDecorator, SafetyCheckDecorator
Iterator	Aggregate	Component
	Concrete Aggregate	Composite
	Iterator	WorkIterator
	Concrete Iterator	CompleteDfsIterator, ActiveTaskIterator

Composite Pattern (Hierarchical Uniformity)

- **3-Level Nesting:** Structure descends through Emergency (Root) → Department (Composite Level 1) → Unit (Composite Level 2) → EmergencyTask (Leaf Level 3)12.
- **Uniform Interface:** Exposes a shared Component base interface (executeResponse(), print()), allowing clients to manipulate individual leaf tasks and multi-level composite branches identically without conditional type checks3more_horiz.
- **Recursive Processing:** Operations like resource distribution or status reporting sweep recursively through the nested tree, leveraging runtime polymorphism to evaluate child nodes67.

State Pattern (Leaf-Level Lifecycles)

- **Leaf-Level Aggregation:** Encapsulates individual task lifecycles by aggregating the abstract EmergencyState directly at the leaf level (EmergencyTask), allowing each task to progress through its lifecycle independently2more_horiz.

- **Decentralized Transitions:** Delegates transition rules and successor designations directly to concrete state subclasses (AlertState, ResponseState, RecoveryState), keeping the context class highly cohesive¹⁰¹¹.
- **Polymorphic Dispatch:** Replaces monolithic switch or if/else control blocks with polymorphic delegation, where the context simply forwards calls to the current state pointer¹²¹³.

Decorator Pattern (Dynamic & Stackable Enhancement)

- **Polymorphic Wrapping:** The abstract Decorator base class inherits from Component while holding a polymorphic reference pointer to an inner wrapped Component^{*414}.
- **Stacked Responsibilities:** Allows runtime attachment of additional, nested behaviors (PriorityDecorator and SafetyCheckDecorator) without subclass-per-combination class bloat^{2more_horiz}.
- **Subtree Decoration:** Because decorators conform to the Component interface, they can wrap individual leaf EmergencyTasks or entire Composite department subtrees, applying behavior recursively to all descendants in a single step¹⁶¹⁷.

Iterator Pattern (Decoupled Aggregate Traversals)

- **Separation of Concerns:** Traversal algorithms are completely decoupled from aggregate maintenance, keeping aggregate interfaces clean and minimizing the likelihood of cascading code changes^{18more_horiz}.
- **Dual Traversal Strategies:** Supports multiple independent, simultaneous traversals over the same tree aggregate:
 - **CompleteDfsIterator:** Explores the entire recursive tree sequentially using an internal depth-first search (DFS) stack^{2more_horiz}.
 - **ActiveTaskIterator:** Evaluates the tree using a custom breadth-first search (BFS) queue to filter out group containers and return only active leaf tasks²²⁰.

Memory & Ownership Policies

- **Polymorphic Safety:** Declares a virtual ~Component() destructor on the polymorphic base class, which is vital to prevent memory leaks and ensure the correct destructor cascades during deletion^{1more_horiz}.
- **Cascading Destruction:** The Composite class takes strict ownership of its added children, recursively deleting them upon its own destruction to clear the entire heap-allocated tree cleanly from memory^{1more_horiz}.
- **Snapshot Traversal Policy:** To prevent active iterator invalidation, segmentation faults, or synchronization conflicts during structural changes (e.g., transferring or deleting tasks), the iterator copies the aggregate state into an internal collection upon instantiation^{24more_horiz}.

Task 3

Traversal modification policy

The system deliberately uses two policies:

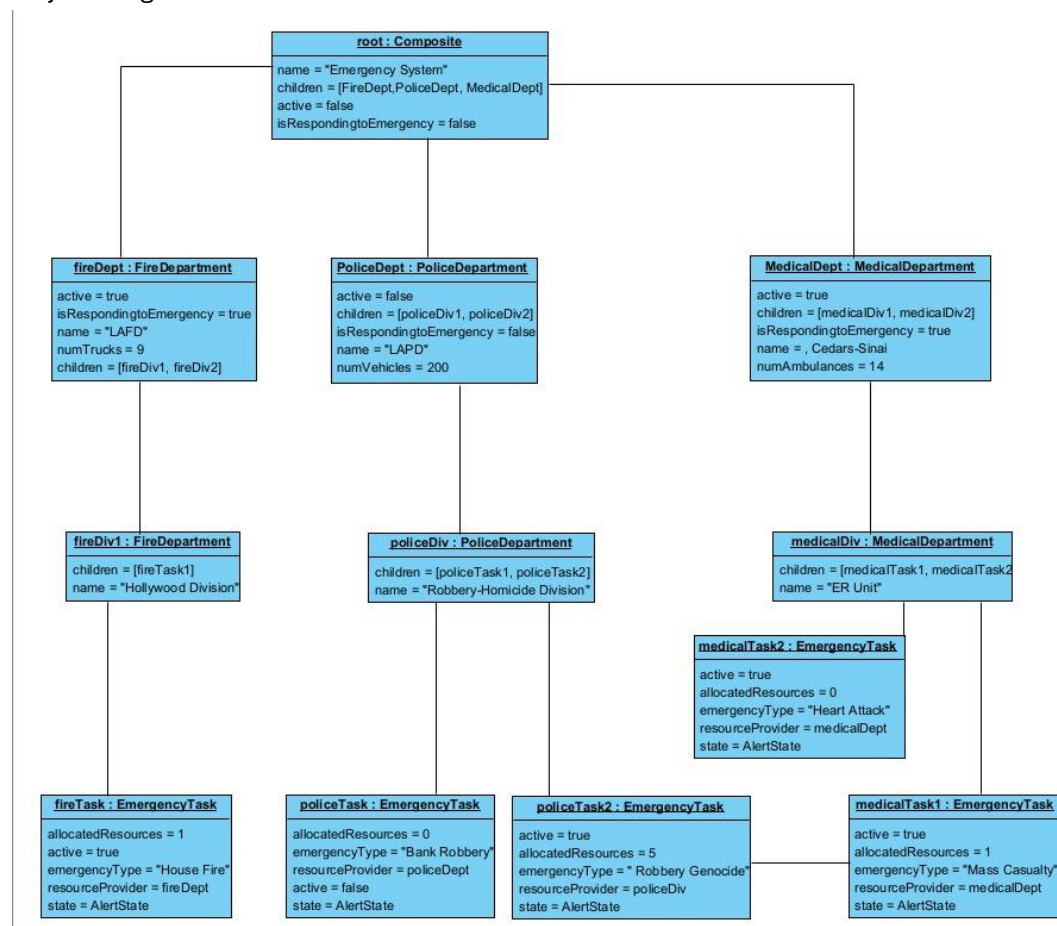
- `CompleteDfsIterator` is live. It walks the hierarchy using its traversal stack and reads current children as it advances. Demonstrations should request a new traversal after structural changes.
- `ActiveTaskIterator` is a snapshot. Its active-leaf queue is built when `first()` is called or when the iterator is constructed. Later additions or removals do not appear in that iterator. Create a new `ActiveTaskIterator` for a fresh snapshot.

The CLI demonstrates this policy by running `status` and `active` again after `newtask`

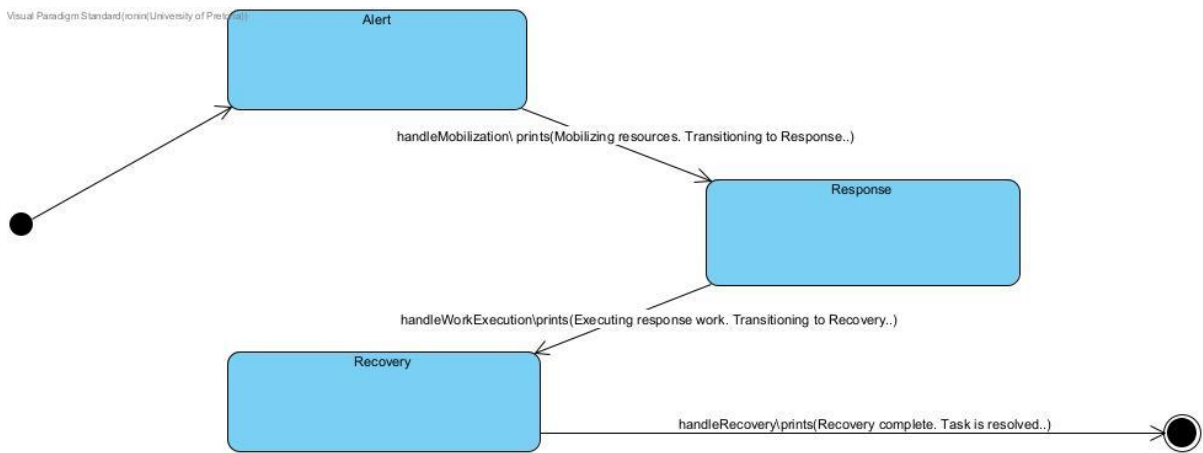
Task 4

Check the attached images/vpp files for a better view in /docs

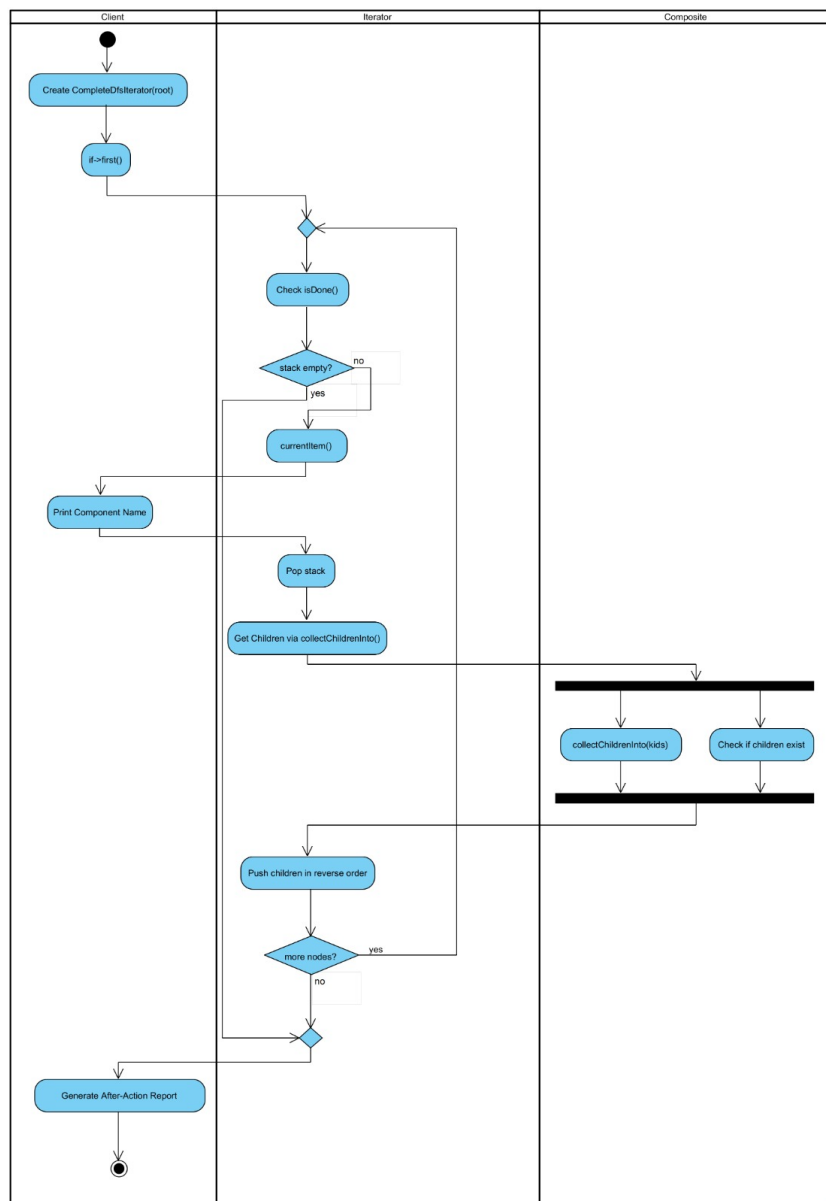
Object Diagram-



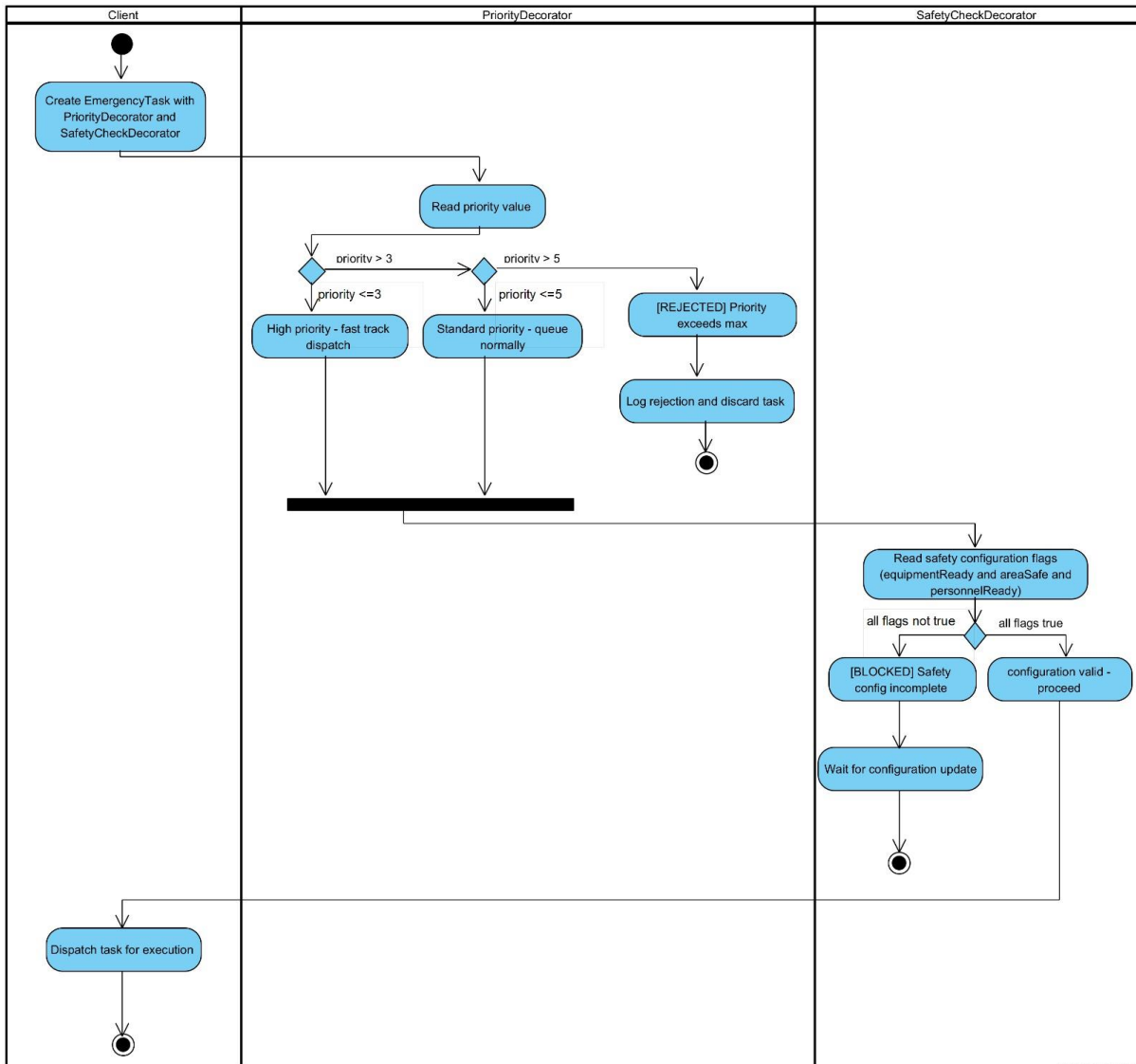
State diagram-



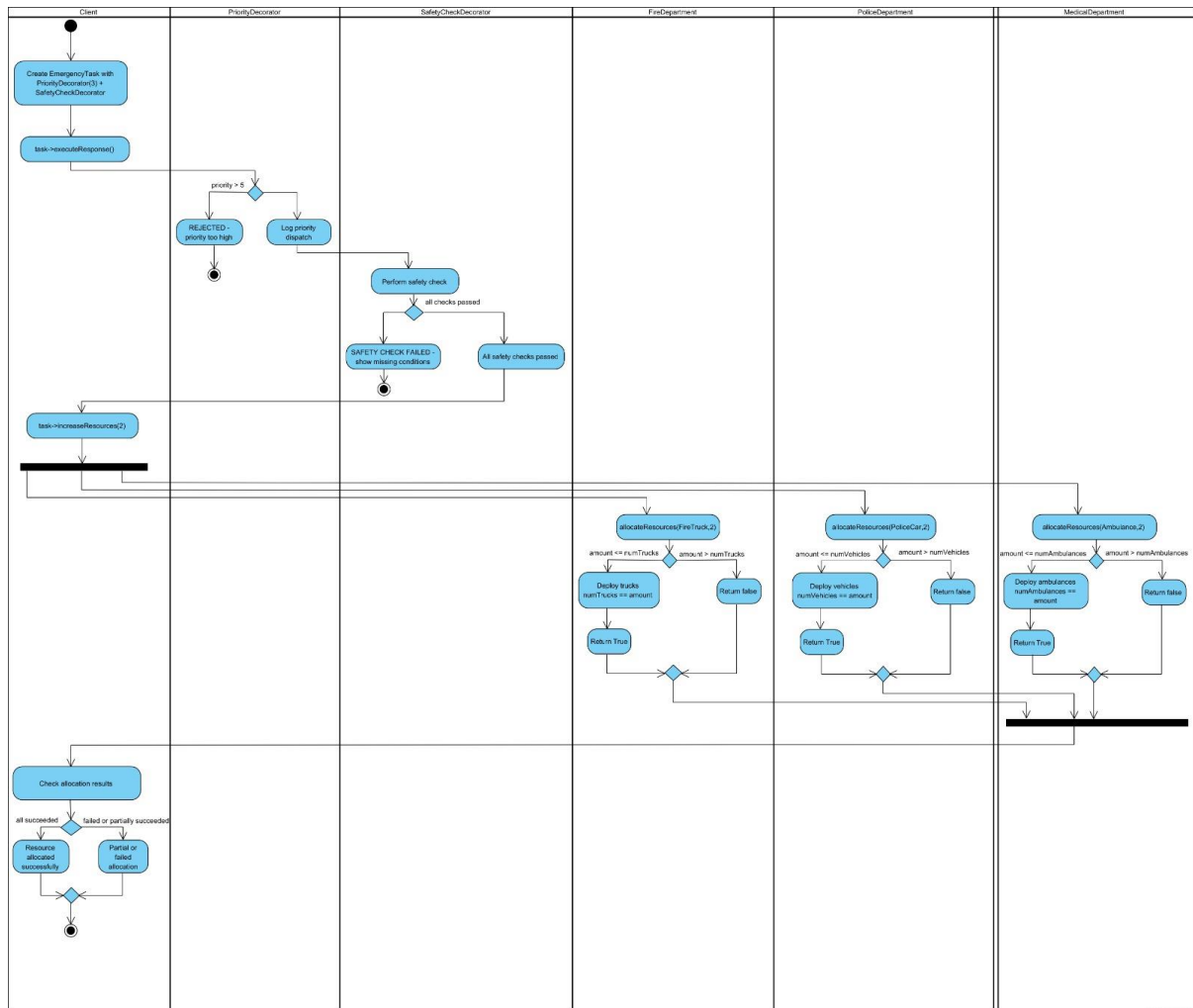
Activity Diagram 1-



Activity Diagram 2-



Activity Diagram 3



Task 5

Valgrind Output

```
State: Alert
Resources: 1
Active: Yes
===== FIRE RESPONSE =====
The Los Angeles Fire Department is responding to a fire alert
1 trucks have been deployed
Composite: Emergency System
Active: No
Responding: No
Number of children: 3
Printing children:
===== FIRE RESPONSE =====
The Los Angeles Fire Department is responding to a fire alert
1 trucks have been deployed

===== POLICE RESPONSE =====
The Los Angeles Police Department is responding to a police alert
0 police cars have been deployed

===== MEDICAL RESPONSE =====
The Cedars-Sinai Medical Center is responding to a medical alert
1 ambulances have been deployed

TEST 7: PASSED

=====
ALL TESTS PASSED SUCCESSFULLY!
=====
==2385==
==2385== HEAP SUMMARY:
==2385==    in use at exit: 0 bytes in 0 blocks
==2385==   total heap usage: 105 allocs, 105 frees, 78,777 bytes allocated
==2385==
==2385== All heap blocks were freed -- no leaks are possible
==2385==
==2385== For lists of detected and suppressed errors, rerun with: -s
==2385== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
motlatso@Theophilus:/mnt/c/Users/Motlatso/COS214/COS214-Practicals/Practical4/COS214_Prac4$ ;93R;97R;97R
```

Using GDB to debug an error encountered

```
75 cout << "Children: " << root->getChildren().size() << " departments\n";
76
77 // Test add/remove
78 EmergencyTask* extraTask = new EmergencyTask("Car Fire", fireDept, Reso
79 fireDept->add(extraTask);
80 cout << "Added 'Car Fire' to Fire Dept (now " << fireDept->getChildren
81 fireDept->remove(extraTask);
82 cout << "Removed 'Car Fire' (now " << fireDept->getChildren().size() <<
83 delete extraTask;
84
85 cout << "\nTEST 1: PASSED\n" << endl;
86
```

Starting program: /mnt/c/Users/Motlatso/COS214/COS214-Practicals/p

```
===== EMERGENCY RESPONSE SYSTEM TESTS =====

TEST 1: Building Composite Structure
-----
Root: Emergency System
Children: 3 departments
Added 'Car Fire' to Fire Dept (now 2 tasks)
Removed 'Car Fire' (now 2 tasks)

Program received signal SIGSEGV, Segmentation fault.
0x00005555555557e94 in main () at main.cpp:83
83      delete extraTask;
(gdb) bt
#0  0x00005555555557e94 in main () at main.cpp:83
(gdb) █
```

We encountered a segmentation fault backtraced it to the exact point of failure and it was our main at line 83 which was deleting out extratask variable of type EmergencyTask

Our destructor had a problem so we were getting a double delete during runtime execution.

Contributions

Ronin: State Diagram, State and Iterator Design, Iterator Implementation, Task 3 main editing, Task 6 ReadME and Dockerfile,

Dingaethu: Activity Diagrams, Decorator and State implementation, Github, pdf doc set up

Motlatso: Object Diagram, Composite and Decorator Design, Composite Implementation and main.cpp, Task 5 debugging

<Repository> https://github.com/Dingalethu/COS214_Prac4/tree/main